# WorksheetDownload.com

10 CONCEPTUAL Q&A  BLOCKCHAIN/WEB3 · PRACTICE WORKSHEET

CONCEPTUAL SERIES
**Smart Contracts**

Name: ________________ Cohort/Batch: ________________ Date: ____________ Score: ________

## SMART CONTRACTS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A  Blockchain/Web3

TOTAL: 10 QUESTIONS

**Q1** What is Smart Contracts and what AI/ML use case is it designed for? *Threat Model*

**Q2** What are the core abstractions in Smart Contracts? *Architecture*

**Q3** How does Smart Contracts handle training or inference? *Configuration*

**Q4** How does Smart Contracts manage data preprocessing? *Hardening*

**Q5** How does Smart Contracts support GPU/TPU acceleration? *SSO / IdP*

**Q6** How do you evaluate a model's performance in Smart Contracts? *Observability*

**Q7** How do you deploy a model trained with Smart Contracts? *Lifecycle*

**Q8** How does Smart Contracts handle distributed or multi-GPU training? *Compliance*

**Q9** How do you track experiments and versions in Smart Contracts? *Testing*

**Q10** What are common pitfalls when using Smart Contracts in production? *Tuning*

# ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

### A1 Smart Contracts is a framework or service

Mitigation

Smart Contracts is a framework or service targeting a specific AI/ML domain training neural networks, building LLM pipelines, deploying models, or orchestrating agents. Understanding its scope helps you know when to use it vs alternatives.

### A2 Smart Contracts organizes work around abstractions like

Primitives

Smart Contracts organizes work around abstractions like models, tensors, pipelines, agents, or embeddings. Learning these core types is the first step to using the framework effectively.

### A3 For training, Smart Contracts defines a model

Hardening

For training, Smart Contracts defines a model architecture, a loss function, and an optimizer. For inference, a trained model processes input data and returns predictions. Batch processing and hardware acceleration are key performance levers.

### A4 Smart Contracts provides data loaders, tokenizers, or

Gotchas

Smart Contracts provides data loaders, tokenizers, or pipeline components that transform raw data into the tensor or embedding format the model expects. Proper preprocessing is critical for model correctness and training speed.

### A5 Smart Contracts detects available accelerators and moves

Federation

Smart Contracts detects available accelerators and moves computation to them. Specify the device explicitly (cuda, mps, tpu) to avoid accidentally running expensive operations on CPU. Mixed-precision (fp16/bf16) further reduces memory and increases throughput.

### A6 Evaluation uses a held-out validation set and

Observability

Evaluation uses a held-out validation set and metrics (accuracy, F1, BLEU, perplexity) appropriate to the task. Monitor both training and validation metrics to detect overfitting. Use a test set only at the very end to report final results.

### A7 Export the model to a portable format

Automation

Export the model to a portable format (ONNX, TorchScript, SavedModel). Serve it via a model server (TorchServe, TF Serving, Triton) or embed it in an API endpoint. Monitor inference latency and drift in production.

### A8 Smart Contracts provides data-parallel or model-parallel

Governance

Smart Contracts provides data-parallel or model-parallel strategies to split work across GPUs. Data parallelism replicates the model on each GPU and averages gradients. Model parallelism splits layers across devices for models too large for one GPU.

### A9 Use an experiment tracker (MLflow, Weights &

Assessment

Use an experiment tracker (MLflow, Weights & Biases, Comet) alongside Smart Contracts to log hyperparameters, metrics, and artifacts. Track the code version and data version for full reproducibility.

### A10 Common issues include training-serving skew (different

Optimization

Common issues include training-serving skew (different preprocessing), missing input validation, no latency SLA enforcement, models not versioned, and no process to retrain on drifted data distributions.